

SASTRA DEEMED UNIVERSITY

School of Computing

DAA301 – Design and Analysis of Algorithms

AI Scribe Generated Answer Sheet

Session ID: 40a852ee-95c4-44d0-a4ef-02cddb4a1292 | Submitted: 2026-06-13 22:09:17

Q1. [2 marks]

Define asymptotic efficiency of an algorithm.

by testing The asymptotic efficiency of an algorithm basically defines how well an algorithm performs in terms of the time taken to complete the given task or the given compute. It basically refers to how fast an algorithm works.

Q2. [2 marks]

What are the steps involved in Divide and Conquer approach?

In a divide and conquer approach, normally we approach the solution by dividing it into smaller parts. That is, we divide it into smaller sub-problems and we keep continuing this process until we reach a problem which cannot be divided further. This is our base problem. We then solve the base problems and then use the solution of the base problem in a bottom-up approach to solve the sub-problems which are basically composed of lower base problems. And using this bottom-up approach which is the conquer part we eventually reach the solution for the main given problem. This is the divide and conquer approach. Read this question.

Q3. [2 marks]

What is meant by a stable sort? Give one example.

[No answer provided]

Q4. [10 marks]

Explain why a reverse sorted array results in worst case time complexity for Insertion Sort. Illustrate with an example.

[No answer provided]

Q5. [10 marks]

What is the Maximum Subarray Problem? Describe the Divide and Conquer approach to solve it and analyse its time complexity.

The maximum subarray problem basically asks us to find the subarray whose sum is maximum. That is given an array what subarray returns the maximum sum. This is what the maximum subarray problem is. To achieve the divide and conquer approach we split the given array continuously from the middle and then we calculate the sums which is obtained from the left hand side, the right hand side and the cross sum and then for each calculation we maintain a pointer which shows the left and right edges of the subarray and during compute what we return what we return is the left and right pointers for whatever sum is maximum. That is if the left sum is maximum, we return the low and high for left sum. If right sum is maximum, we return the low and high for right sum. And if cross sum is maximum, then we return the low and high for cross sum. We recursively do this in a bottom-up approach and that

ultimately gives us the maximum very for the entire problem.